Algorithms for Data Science

Lecture #5: Sorting

Mateusz Buczyński

University of Warsaw

25.03.2026

Thanks to dr Krzysztof Fleszar for the base material.

Sorting: Problem Statement

Input

- An array $a[1..n]$

Output

- The array is sorted in nondecreasing order:

$$a[1] \leq a[2] \leq \cdots \leq a[n]$$

Practical note

- Sorting is needed very often.
- In practice, a sorting routine is usually already implemented (library function).

A Surprising Fact About Sorting

Theorem (comparison model)

Any comparison-based sorting algorithm requires

$$\Omega(n \log n)$$

comparisons in the worst case.

Important caveat

This lower bound applies only to algorithms that learn order *through comparisons* (e.g., "$a_i < a_j$").

- So no comparison sort can be worst-case $O(n)$ or $O(n \log \log n)$.
- Efficient comparison sorts (merge sort, heap sort) are asymptotically optimal.

Why $\Omega(n \log n)$?

Setup (assume all elements are distinct)

- There are $n!$ possible input orders (permutations).
- Exactly one permutation is the correct sorted order for a given input.

Key counting idea

- One comparison has 2 outcomes, so it can eliminate at most about half of the remaining possibilities.
- After k comparisons, at most 2^k cases can be distinguished.
- To distinguish all $n!$ cases, we need

$$2^k \geq n! \quad \Rightarrow \quad k \geq \log_2(n!).$$

Conclusion

$$k \geq \log_2(n!) \approx \log_2(n^n) = n \log_2 n \quad \Rightarrow \quad k = \Omega(n \log n).$$

Insertion Sort: Visual Idea

Grow a sorted prefix

At each step, take the next element and insert it into the already sorted left part.

Example

$$[4 \mid 1, 5, 3]$$

$$[1, 4 \mid 5, 3]$$

$$[1, 4, 5 \mid 3]$$

$$[1, 3, 4, 5]$$

What to notice

- Left side is always sorted.
- One new element is inserted each round.
- Good intuition: like sorting cards in your hand.

Insertion Sort: One Insertion Step

Input example

$$a = [4,\ 1,\ 5,\ 3,\ 6,\ 2]$$

Teaching idea

At this moment of insertion sort, we treat the last value as the key and insert it into the already sorted prefix.

Decision rule

For key $= 2$, scan from right to left in the sorted part:

$$[1,\ 3,\ 4,\ 5,\ 6]$$

Shift every element > 2 one position to the right, then place 2 into the first free position.

Insertion Sort: Result of Inserting 2

Before insertion

$[1,\ 3,\ 4,\ 5,\ 6,\ 2]$

After insertion

$[1,\ 2,\ 3,\ 4,\ 5,\ 6]$

What changed?

- $6, 5, 4, 3$ were shifted one cell to the right.
- 2 was inserted between 1 and 3.

Insertion Sort: Complexity at a Glance

Core properties

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: Yes

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: Yes

Best case

- Input already sorted
- Time: $\Theta(n)$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: Yes

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \cdots + n$$

- Time:

$$\Theta(1 + 2 + \cdots + n) = \Theta(n^2)$$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: Yes

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \cdots + n$$

- Time:

$$\Theta(1 + 2 + \cdots + n) = \Theta(n^2)$$

Average case

$$\Theta(n^2)$$

Insertion Sort: Complexity at a Glance

Core properties

- Time (worst case): $\Theta(n^2)$
- Extra space: $\Theta(1)$
- Stable: Yes

Best case

- Input already sorted
- Time: $\Theta(n)$

Worst case

- Input sorted in reverse order
- Work per step forms

$$1 + 2 + \cdots + n$$

- Time:

$$\Theta(1 + 2 + \cdots + n) = \Theta(n^2)$$

Average case

$$\Theta(n^2)$$

Practical note

Insertion sort is often fast in practice for small n.

Why "Stable" Matters

Definition (stability)

A sorting algorithm is stable if equal elements keep their original relative order.

Input with labels

$$[1_A,\ 2,\ 1_B]$$

(1_A appears before 1_B)

After stable sort

$$[1_A,\ 1_B,\ 2]$$

Order of equal keys is preserved.

If not stable

$$[1_B,\ 1_A,\ 2]$$

This changes the order of equal keys.

Insertion sort

Insertion sort is stable, because insertion shifts larger elements right and does not swap equal keys past each other.

Merge Sort: Visual Idea

Split, sort, merge

Break the array into smaller parts, sort them recursively, then merge sorted parts back together.

Example

$$[4,1,5,3]$$

$$[4,1] \qquad [5,3]$$

$$[1,4] \qquad [3,5]$$

$$[1,3,4,5]$$

What to notice

- The problem is divided into smaller subproblems.
- Merging two sorted lists is linear.
- The structure is very regular.

Merge Sort: Idea and Workflow

Core properties

- Paradigm: divide & conquer
- Stable:

Merge Sort: Idea and Workflow

Core properties

- Paradigm: divide & conquer
- Stable: Yes (equal elements keep relative order)
- Extra space:

Merge Sort: Idea and Workflow

Core properties

- Paradigm: divide & conquer
- Stable: Yes (equal elements keep relative order)
- Extra space: $O(n)$

Merge Sort: Running Time

Recurrence

For $n > 1$, merge sort performs:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

- $2T(n/2)$: recursively sort two halves
- $\Theta(n)$: merge step

Best case = Worst case

- The algorithm always splits into halves the same way.
- Merging still scans all elements, regardless of initial order.

$$T(n) = \Theta(n \log n)$$

At a glance

Time: $\Theta(n \log n)$, Space: $O(n)$, Stable: Yes

Quick Sort: Visual Idea

Partition around a pivot

Choose one element as a pivot, place smaller elements on one side and larger elements on the other, then recurse.

Example

$$[4, 1, 5, 3]$$

$$[1, 3] < 4 < [5]$$

$$[1, 3] \rightarrow [1, 3]$$

$$[1, 3, 4, 5]$$

What to notice

- The pivot lands in its final position.
- Only the left and right parts remain to be sorted.
- Performance depends on partition balance.

Practical note

Quick sort is often very fast in practice for large n.

Quick Sort: Complexity and Caveats

At a glance

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Extra space: $O(\log n)$

Quick Sort: Complexity and Caveats

At a glance

Best case: $\Theta(n \log n)$

Average case: $\Theta(n \log n)$

Worst case: $\Theta(n^2)$

Extra space: $O(\log n)$

Stable: No

Why worst case can be quadratic

If partitioning is very unbalanced (e.g., pivot always smallest/largest), recursion depth becomes n:

$$T(n) = T(n-1) + \Theta(n)$$

$$\Rightarrow \Theta(n + (n-1) + \cdots + 1) = \Theta(n^2).$$

Typical bad pattern

With pivot = first element and reverse input

$$[7, 6, 5, 4, 3, 2, 1],$$

each step removes only one element

Data Structures for Dynamic Sets

Notation

n = current number of stored elements.

Why structure matters

A data structure keeps an invariant (or not), and that directly determines operation costs.

Structure	**add**	**find**	**find max**	**remove max**
Unsorted array (no invariant)	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Binary heap (heap property)	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$
Sorted array (sorted invariant)	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$	$\Theta(1)$

Takeaway

There is no universally best structure: performance depends on which operations are most frequent.

How the Invariant Changes the Cost

Unsorted array

- Add at end: $\Theta(1)$
- To find max, scan all: $\Theta(n)$
- Remove max needs scan first

Binary heap

- Max kept at root: $\Theta(1)$
- Add/remove via sift up/down: $\Theta(\log n)$
- Arbitrary find still $\Theta(n)$

Sorted array

- Binary search: $\Theta(\log n)$
- Max at one end: $\Theta(1)$
- Add needs shifts: $\Theta(n)$

Decision rule

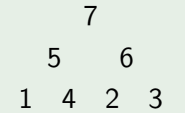
If you need fast `find max` and `remove max` with many updates, a binary heap is usually the best compromise.

What Does a Heap Look Like?

Binary heap = complete binary tree

It is usually stored in an array, but we can visualize it as a nearly full binary tree.

Tree view (max-heap)



Each parent is at least as large as its children.

Array view

$$[7, 5, 6, 1, 4, 2, 3]$$

- Root at index 1
- Children of index k: $2k$ and $2k + 1$
- Parent of index k: $\lfloor k/2 \rfloor$

Important

A heap is **not** a sorted array. It guarantees only local order: parent vs. children.

Heap Sort via a Binary Heap

Key idea

Maintain a max-heap as the data structure.

Invariant

Heap property: every parent key is $\geq$ each child key.

- Let n be the maximum number of elements stored.
- This invariant is what makes operations efficient.

Binary Heap Operation Costs

Running times

- **add**: $O(\log n)$
- **find max**: $\Theta(1)$
- **remove max**: $O(\log n)$

Why this is useful for heap sort

Sorting repeatedly uses **remove max**, so we rely on the $O(\log n)$ bound while max lookup stays constant time.

Heap Sort: Idea and Heap Operations

Invariant

Heap property is maintained at all times (max-heap).

Binary Heap Operations

- `add(x)`: $O(\log n)$
- `findMax()`: $\Theta(1)$
- `removeMax()`: $O(\log n)$

Heap Sort Strategy

- Build a max-heap from all input elements.
- Repeatedly extract maximum.
- Write extracted values from right to left.

Heap Sort Algorithm and Cost

Input / Output

Input: array $a[1..n]$
Output: a is sorted (nondecreasing order)

Algorithm (using auxiliary heap H)

$H \leftarrow$ new empty max-heap

for $i = 1$ to n : H.add($a[i]$)

for $i = n$ to 1 : $a[i] \leftarrow H$.findMax(), H.removeMax()

Running Time

- Build phase: $n \cdot O(\log n) = O(n \log n)$
- Extract phase: $n \cdot (\Theta(1) + O(\log n)) = O(n \log n)$
- Total: $\Theta(n \log n)$ (best and worst case)

Properties

- Extra space: $\Theta(n)$
- Stable: No

Heap Sort Optimization: Build Heap In-Place

Main idea

Do not create an auxiliary heap. Treat the input array itself as a max-heap.

In-place initialization

$$H \leftarrow \texttt{make\_heap}(a)$$

Time: $\Theta(n)$

Why this matters

- We avoid storing all elements in a separate heap.
- Heap construction is linear, not $n \cdot O(\log n)$.
- The heap invariant is preserved on the active prefix of the array.

Optimized Heap Sort: Algorithm and Final Costs

Algorithm (in-place)

Input: array $a[1..n]$
Output: a sorted in nondecreasing order

$$H \leftarrow \texttt{make\_heap}(a)$$

$$\text{for } i = n \text{ downto } 1: \quad H.\texttt{removeMax()}$$

Running time

- Build heap: $\Theta(n)$
- n extractions: $n \cdot O(\log n) = O(n \log n)$
- Total: $\Theta(n \log n)$ (best case = worst case)

Properties

- Extra space: $\Theta(1)$
- Stable: No

Comparison and Practice

Stable sorting

A sorting algorithm is stable if equal elements remain in their original relative order.

Algorithm	Time	Extra space	Stable
Insertion Sort	$\Theta(n^2)$	$\Theta(1)$	Yes
Merge Sort	$\Theta(n \log n)$	$\Theta(n)$	Yes
Quick Sort	$\Theta(n^2)$ worst case	$\Theta(\log n)$	No
Heap Sort	$\Theta(n \log n)$	$\Theta(1)$	No

Practical reading of the table

- Insertion Sort is simple and good for small inputs.
- Merge Sort gives predictable $\Theta(n \log n)$ time and stability.
- Quick Sort is often very fast in practice, but has a quadratic worst case.
- Heap Sort keeps $\Theta(n \log n)$ time with only constant extra memory.

Heap Representation in an Array

One-based indexing (max-heap stored in $a[1..i]$)

$a[1]$ $a[2]$ $a[3]$ $a[4]$ $a[5]$ $a[6]$ $a[7]$ $\qquad (i = 7)$

Same structure as a binary tree

index 1

indices 2, 3

indices 4, 5, 6, 7

- Root is at index 1.
- Children of index 2: $4, 5$.
- Children of index 3: $6, 7$.

Why this is useful

No pointers are needed:

- Tree navigation is done by index arithmetic.
- Heap operations (`add`, `removeMax`) move along parent/child indices.

Heap Index Formulas (1-Based)

For a node at index k

$$\text{parent}(k) = \left\lfloor \frac{k}{2} \right\rfloor, \qquad \text{left}(k) = 2k, \qquad \text{right}(k) = 2k + 1$$

Concrete example: $k = 3$

$$\text{parent}(3) = 1$$

$$\text{left}(3) = 6$$

$$\text{right}(3) = 7$$

Boundary rule

With current heap size i:

- A child exists only if its index is $\leq i$.
- If $2k > i$, node k is a leaf.

Heap Example: Valid Case

Max-Heap Property (array view)

For all indices k with $1 < k \leq i$:

$$a[k] \leq a[\lfloor k/2 \rfloor]$$

Example a with $i = 7$

$$a[1..7] = [7,\ 5,\ 6,\ 1,\ 4,\ 2,\ 3]$$

- Parent of index k is at $\lfloor k/2 \rfloor$.
- Every child value is $\leq$ its parent value.

So this is a heap.

Heap Example: Non-Heap Case

Counterexample b

$$b[1..7] = [7,\ 5,\ 4,\ 6,\ 3,\ 2,\ 1]$$

Check index $k = 4$:

$$b[4] = 6, \qquad \lfloor 4/2 \rfloor = 2, \qquad b[2] = 5$$

But $6 \not\leq 5$, so the heap property is violated.

Decision Rule

A max-heap requires:

$$\forall k\ (1 < k \leq i):\ a[k] \leq a[\lfloor k/2 \rfloor]$$

If one index fails, it is not a heap.

Heap Example: Active Heap Prefix

Max-heap property (1-based indexing)

For all indices k with $1 < k \leq i$:

$$a[k] \leq a\left[\left\lfloor \frac{k}{2} \right\rfloor\right]$$

Given array and heap size

$$a[1..7] = [6,\ 5,\ 4,\ 3,\ 2,\ 1,\ 99], \qquad i = 6$$

- The current heap is only $a[1..i] = a[1..6]$.
- $a[7] = 99$ is outside the active heap.

Why This Is Still a Heap

Tree view of $a[1..6]$

$$\begin{array}{ccccc} & & 6 & & \\ & 5 & & 4 & \\ 3 & & 2 & & 1 \end{array}$$

Checks $(1 < k \leq 6)$

$$5 \leq 6, \quad 4 \leq 6,$$
$$3 \leq 5, \quad 2 \leq 5, \quad 1 \leq 4$$

Decision

Yes, this is a heap for $i = 6$. Only indices $1..i$ are tested.

Heap: Operations (Max-Heap)

Heap property

For every node except the root:

$$a[k] \leq a\left[\left\lfloor \frac{k}{2} \right\rfloor\right], \qquad 1 < k \leq i$$

So each child is at most its parent.

Array view (example)

index	1	2	3	4	5	6
a[index]	6	5	4	3	1	9

Indices map to a complete binary tree: parent of k is $\lfloor k/2 \rfloor$, children are $2k$ and $2k+1$.

Find maximum

In a max-heap, the maximum is always at the root:

`return` $a[1]$

Heap Operations: Add and Remove Max

Add(v) (sift up)

- Put v in the next free slot: $a[i+1] \leftarrow v$, then $i \leftarrow i+1$.
- While parent is smaller, swap with parent.
- Stop when heap property is restored.

Key idea

Insertion may move the new value upward until

$$a[\text{parent}] \geq a[\text{child}].$$

Remove Max (sift down)

- Swap $a[1]$ with $a[i]$ (move max to the end).
- Decrease heap size: $i \leftarrow i-1$.
- Starting at $a[1]$, while a larger child exists:
  - swap with the larger child.
- Stop when heap property is restored.

Max-Heap Operations: Find Max and Add

Find Max

In a max-heap, the largest element is at the root:

`return` $a[1]$

$$T(n) = \Theta(1)$$

Add(v)

- Insert at the end: $a[i+1] \leftarrow v,\ i \leftarrow i+1$.
- Restore heap order by Upheap(j):
  - while parent is smaller, swap with parent.

Upheap(j) idea

- Node moves only along the path to the root.
- At most one swap per level.

Cost

$$T_{\texttt{add}}(n) = T_{\texttt{upheap}}(n) = O(\log n)$$

Max-Heap Operation: Remove Max

Remove Max

- Swap $a[1]$ with $a[i]$ (max goes to the end).
- Decrease heap size: $i \leftarrow i - 1$.
- Starting from root, run Downheap(j):
  - while a larger child exists, swap with the larger child.

Downheap(j) cost

$$T_{\texttt{removeMax}}(n) = T_{\texttt{downheap}}(n) = O(\log n)$$

Why $O(\log n)$?

- A binary heap is a complete binary tree.
- With i elements, tree height is $O(\log i)$.
- Upheap/Downheap follow one root-to-leaf path.

Notation note

n = current number of heap elements (here: $n = i$).

Breaking the $\Omega(n \log n)$ Bound

Comparison-based sorting lower bound

If an algorithm relies only on comparisons, then its worst-case time is

$$\Omega(n \log n).$$

Counting Sort: when we can do better

Input: array $a[1..n]$ such that

$$0 \leq a[i] \leq k \quad \text{for all } i.$$

Assumption: k is known (or precomputed).

Counting Sort: Core Procedure

High-level steps

- Count how often each value $0, 1, \ldots, k$ appears.
- Compute cumulative sums (prefix counts).
- Place each element into its correct final position.

Radix Sort: Breaking the Comparison Bound

Key Idea

Use a **stable** linear-time sort (e.g., Counting Sort) multiple times on different digits/fields.

Time per pass: $\Theta(n + k)$

Example: Sort students by birthdate

Represent each date as `Year.Month.Day`.

Algorithm (LSD $\rightarrow$ MSD)

- Sort by **Day**
- Then sort by **Month**
- Then sort by **Year**

Typical key ranges

$\text{Day} \leq 31, \quad \text{Month} \leq 12, \quad \text{Year} \leq 2026$

(so each pass can use Counting Sort)

Why This Order Is Correct

Crucial property

Counting Sort is **stable**: equal keys keep their previous relative order.

- After sorting by Day, ties are unresolved by Month/Year.
- Sorting by Month keeps Day-order inside each month.
- Sorting by Year keeps Month+Day order inside each year.

Important caveat

Sorting in the opposite order (**Year** $\rightarrow$ **Month** $\rightarrow$ **Day**) does **not** produce correct full-date order.

Stalin Sort: A Joke Counterexample

Idea

Scan from left to right and keep only elements that are at least as large as the last kept one. All other elements are simply removed.

Example

$$[3, 1, 4, 2, 5] \quad \Rightarrow \quad [3, 4, 5]$$

because 1 and 2 are "purged".

Why this is not a real sorting algorithm

- Output may have fewer elements than input.
- It does not produce a permutation of the original array.
- So it solves a different problem.

Teaching note

Like Bogosort, Stalin Sort is mainly a humorous example used to discuss what a sorting

Order Statistics: QuickSelect

Problem

Instead of sorting everything, we may want only the k-th smallest element: minimum, maximum, median, quartile, etc.

Key idea

QuickSelect uses the same partition step as Quick Sort. After partitioning around a pivot, only **one side** can still contain the desired rank.

Consequence

We recurse into only one subproblem, not both.

QuickSelect: Cost and Median

Running time

Best / average: $\Theta(n)$

Worst case: $\Theta(n^2)$

The worst case appears when pivots are repeatedly very unbalanced.

Median

To find the median, ask for rank

$$k = \left\lceil \frac{n}{2} \right\rceil .$$

So the median can usually be found without fully sorting the array.

Comparison with sorting

If we only need one order statistic, QuickSelect is typically better than paying $\Theta(n \log n)$ for a full sort.

External Sorting

When do we need it?

Sometimes the dataset is so large that it does not fit into RAM. Then the main bottleneck is often disk I/O, not CPU time.

Basic strategy

- Load a chunk that fits into memory.
- Sort it internally and write it back to disk as a run.
- Repeat for all chunks.
- Merge the sorted runs.

Main idea

External Sorting is essentially Merge Sort adapted to secondary storage.

External Sorting: Why Merging Works Well

Why merge?

- Runs are already sorted.
- Sequential disk access is much faster than many random reads.
- Multi-way merge can combine many runs at once.

Typical tools

- Buffer pages in RAM
- Priority queue / heap
- Temporary files on disk

Takeaway

For massive datasets, the right question is not only "How many comparisons?", but also "How many passes over disk data?"